

ITELEC 4100
LECTURE # 2

PHP | OOP

1 **Object-Oriented Programming**

OOP in PHP

What is OOP?

- Object-Oriented Programming (OOP) is a programming model that is based on the concept of **classes** and **objects**.
- OOP has several advantages over conventional or procedural style of programming.

Advantages of OOP?

- It provides a clear modular structure for the programs.
- It helps you adhere to the “don’t repeat yourself (DRY) principle, thus make your code much easier to maintain, modify and debug.
- It makes it possible to create methods with less code and shorter development time with high degree of reusability.

Why implement OOP?

- Good structure of the program
- Code reusability
- Faster development
- Easier to maintain
- **Laravel** implements OOP concepts

5

2

Classes and Objects

OOP in PHP

6

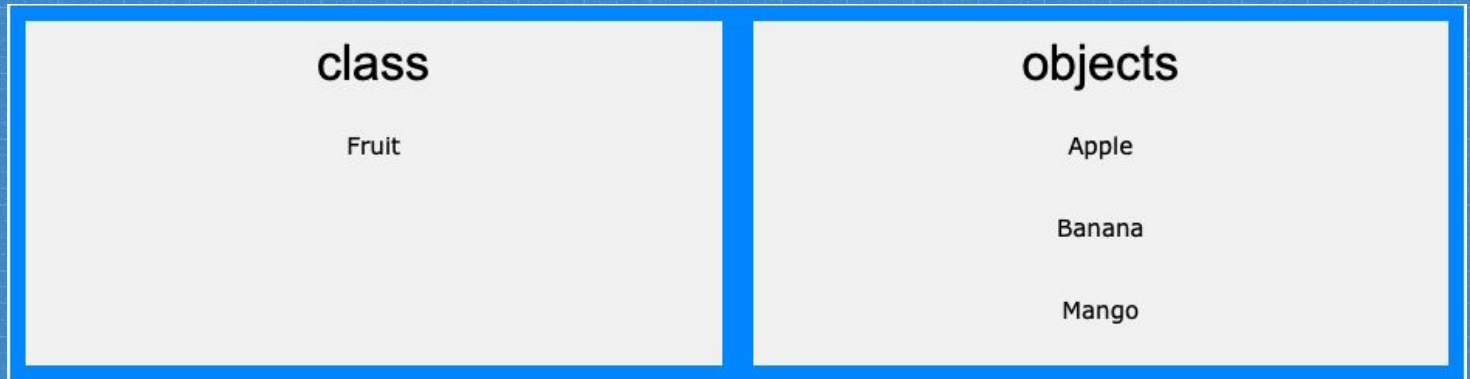
Classes and Objects

- The two main aspects of OOP are classes and objects.
- A **class** is a self-contained, independent collection of variables and functions which work together to perform one or more specific tasks.
- An **object** are individual instances of a class.

Classes and Objects

- A class acts as **template** or **blueprint** from which individual objects can be created.
- When individual objects are created, they inherit the same generic properties and behaviors.
- Note that, each object may have **different values** for **certain properties**.

Classes and Objects



9

Classes and Objects

- For example, think of a class as a blueprint for a **house** while object is like an **actual house built** according to that blueprint.
- We can build several identical houses from the same blueprint, however each house may have **different paints, interiors, and families inside**.

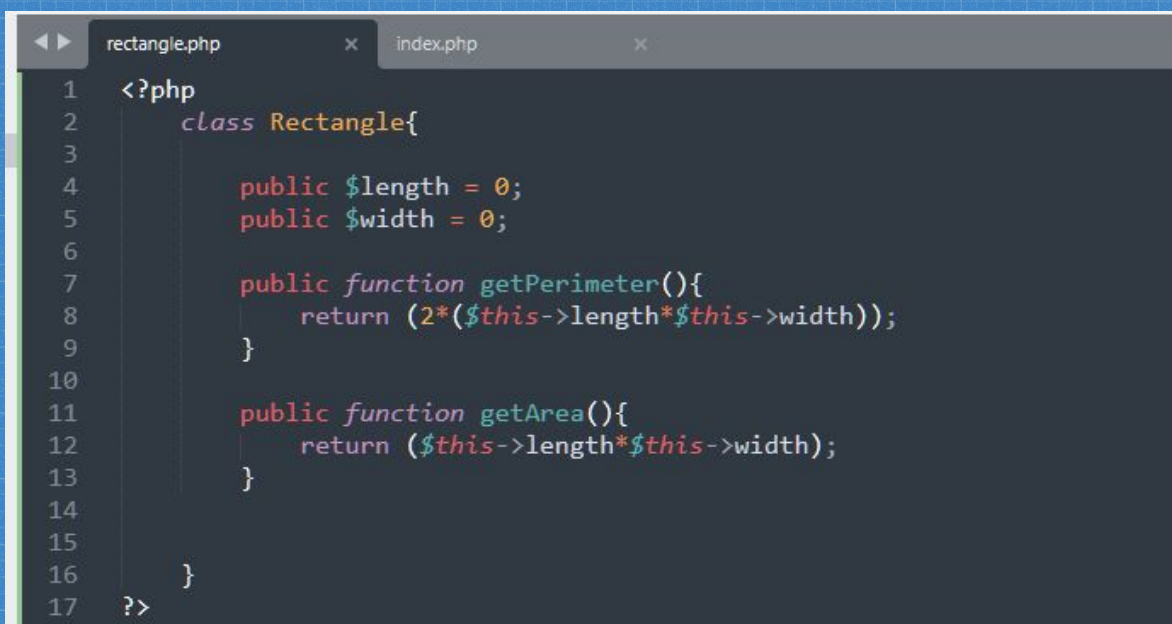
10

Classes and Objects

- A class can be declared using the **class** keyword, followed by the name of the class, and a **pair of curly braces {}**.
- Sample code next, slide.

11

Sample code – rectangle.php

A screenshot of a code editor with two tabs: 'rectangle.php' and 'index.php'. The 'rectangle.php' tab is active, showing PHP code for a Rectangle class. The code includes two public properties, \$length and \$width, both initialized to 0. It also includes two public methods: getPerimeter() which returns 2*(\$this->length*\$this->width), and getArea() which returns \$this->length*\$this->width. The code is syntax-highlighted with colors: blue for keywords, red for property names, and black for other text. Line numbers 1 through 17 are visible on the left side of the editor.

```
1 <?php
2     class Rectangle{
3
4         public $length = 0;
5         public $width = 0;
6
7         public function getPerimeter(){
8             return (2*($this->length*$this->width));
9         }
10
11        public function getArea(){
12            return ($this->length*$this->width);
13        }
14
15    }
16
17 ?>
```

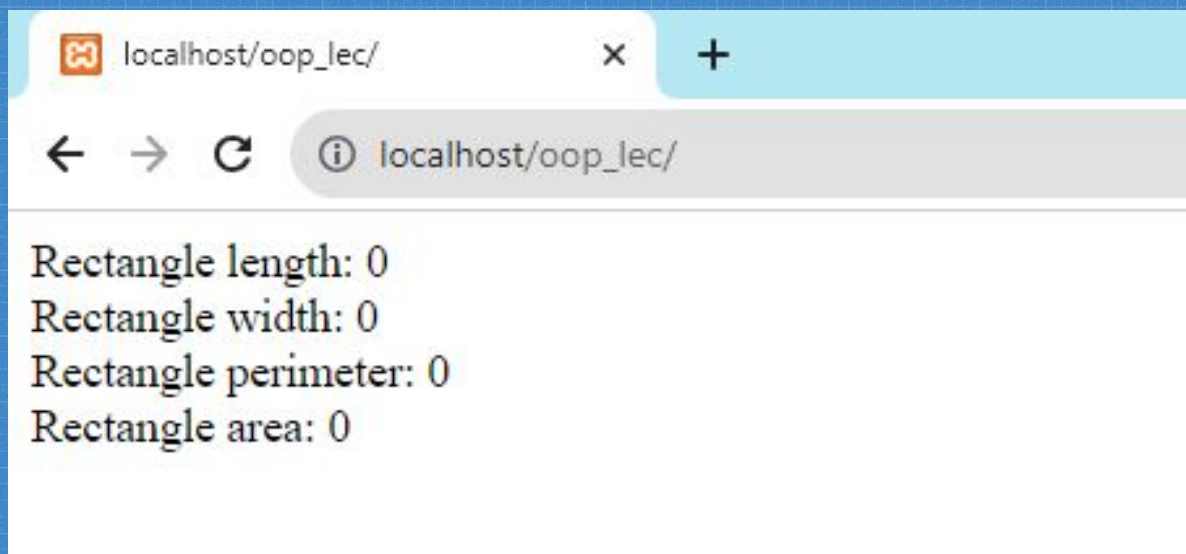
12

Sample code – index.php

```
index.php x rectangle.php x
1 <?php
2     include "rectangle.php";
3
4     $rec_obj = new Rectangle();
5
6     echo "Rectangle length: ".$rec_obj->length."<br>";
7     echo "Rectangle width: ".$rec_obj->width."<br>";
8     echo "Rectangle perimeter: ".$rec_obj->getPerimeter()."<br>";
9     echo "Rectangle area: ".$rec_obj->getArea()."<br><br>";
10
11 ?>
```

13

Output of index.php



A screenshot of a web browser window. The address bar shows 'localhost/oop_lec/'. The page content displays the output of the PHP script: 'Rectangle length: 0', 'Rectangle width: 0', 'Rectangle perimeter: 0', and 'Rectangle area: 0', each on a new line.

localhost/oop_lec/ x +

← → ↻ ⓘ localhost/oop_lec/

Rectangle length: 0
Rectangle width: 0
Rectangle perimeter: 0
Rectangle area: 0

14

Classes and Objects

- Variables within a class are called **properties**, while functions are called **methods**.
- Class names are also conventionally written in **PascalCase** i.e. each concatenated word starts with an uppercase letter (e.g. MyClass).

Classes and Objects

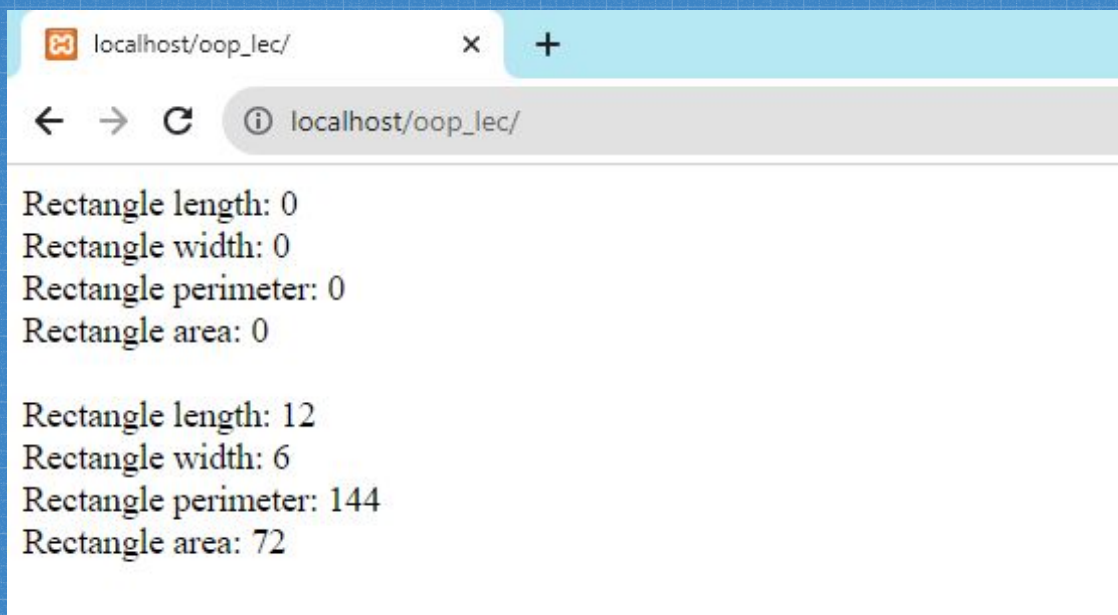
- The **arrow symbol (->)** is an OOP construct that is used to **access** contained properties and methods of a given object.
- The pseudo-variable **\$this** provides a reference to the calling object.
- The full capabilities of OOP becomes evident when using multiple instances of the same class, as shown in the next slide.

Revised sample code – test.php

```
index.php  x  rectangle.php  x
1  <?php
2      include "rectangle.php";
3
4      $rec_obj = new Rectangle();
5
6      echo "Rectangle length: ".$rec_obj->length."<br>";
7      echo "Rectangle width: ".$rec_obj->width."<br>";
8      echo "Rectangle perimeter: ".$rec_obj->getPerimeter()."<br>";
9      echo "Rectangle area: ".$rec_obj->getArea()."<br><br>";
10
11     $rec_obj2 = new Rectangle();
12     $rec_obj2->length = 12;
13     $rec_obj2->width = 6;
14
15     echo "Rectangle length: ".$rec_obj2->length."<br>";
16     echo "Rectangle width: ".$rec_obj2->width."<br>";
17     echo "Rectangle perimeter: ".$rec_obj2->getPerimeter()."<br>";
18     echo "Rectangle area: ".$rec_obj2->getArea()."<br><br>";
19  ?>
20
```

17

Output of revised test.php



localhost/oop_lec/ x +

← → ↻ ⓘ localhost/oop_lec/

Rectangle length: 0
Rectangle width: 0
Rectangle perimeter: 0
Rectangle area: 0

Rectangle length: 12
Rectangle width: 6
Rectangle perimeter: 144
Rectangle area: 72

18

3

Constructor & Destructor

OOP in PHP

19

Constructor

- Constructor allows us to initialize the object's properties upon creation of the object.
- We use `__construct()` function to declare constructor inside the class.
- **Double underscore** (`__`) before the construct.

20

Sample code – rectangle.php

```
1 <?php
2 class Rectangle{
3
4     public $length = 0;
5     public $width = 0;
6
7     function __construct($len,$wid){
8         $this->length = $len;
9         $this->width = $wid;
10    }
11
12    public function getPerimeter(){
13        return (2*($this->length*$this->width));
14    }
15
16    public function getArea(){
17        return ($this->length*$this->width);
18    }
19
20
21 }
22 ?>
```

21

Sample code – index.php

```
1 <?php
2 include "rectangle.php";
3
4 $rec_obj = new Rectangle(10,8);
5
6 echo "Rectangle length: ".$rec_obj->length."<br>";
7 echo "Rectangle width: ".$rec_obj->width."<br>";
8 echo "Rectangle perimeter: ".$rec_obj->getPerimeter()."<br>";
9 echo "Rectangle area: ".$rec_obj->getArea()."<br><br>";
10
11 ?>
```



localhost/oop_lec/

Rectangle length: 10
Rectangle width: 8
Rectangle perimeter: 160
Rectangle area: 80

22

Destructor

- **Destructor** is called when the object is destructed or the script is stopped or exited.
- **__destruct** function will be called at the end of the script.
- **Double underscore** (__) before the destruct.

23

Sample code – rectangle.php



```
1 <?php
2 class Rectangle{
3
4     public $length = 0;
5     public $width = 0;
6
7     function __construct($len,$wid){
8         $this->length = $len;
9         $this->width = $wid;
10    }
11
12    public function getPerimeter(){
13        return (2*($this->length*$this->width));
14    }
15
16    public function getArea(){
17        return ($this->length*$this->width);
18    }
19
20    function __destruct(){
21        echo "Rectangle length: ".$this->length."<br>";
22        echo "Rectangle width: ".$this->width."<br>";
23        echo "Rectangle perimeter: ".$this->getPerimeter()."<br>";
24        echo "Rectangle area: ".$this->getArea()."<br><br>";
25    }
26 }
27 ?>
```

24

Sample code – index.php

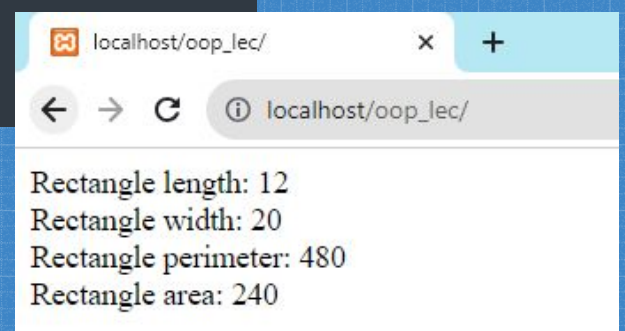
```
index.php  rectangle.php x
1  <?php
2      include "rectangle.php";
3      $rec_obj = new Rectangle(10,8);
4  ?>
5
```



25

Sample code 2 – index.php

```
index.php  rectangle.php x
1  <?php
2      include "rectangle.php";
3      $rec_obj = new Rectangle(10,8);
4
5      $rec_obj->length = 12;
6      $rec_obj->width = 20;
7  ?>
8
```



26

Constructor & Destructor tips!

- **Constructor** is invoked automatically every time you create an object.
- **Destructor** is invoked automatically every time you create a `__destruct()` function.
- As constructors and destructors helps reducing the amount of code, they are very useful!

27

4

Access Modifiers

OOP in PHP

28

Access Modifiers

- When working with classes, you can even restrict access to its properties and methods using the **visibility keywords** (access modifiers) for greater control.
- There are **three** visibility keywords which determines how and from where properties and methods can be accessed and modified

29

Access Modifiers

- **public** – a public property or method can be accessed anywhere within or outside of class. Default in PHP
- **protected** - a protected property or method can only be accessed within the class itself or in inherited classes.
- **private** – a private property or method is accessible only within the class that defines it.

30

5

Inheritance

OOP in PHP

31

Inheritance

- Classes can inherit the properties and methods of another class using the ***extends*** keyword.
- It is probably the most powerful reason behind using the object-oriented programming model.

32

Sample code – fruits.php

```
1 <?php
2 class Fruit {
3     public $name;
4     public $color;
5
6     public function __construct($nm,$clr){
7         $this->name = $nm;
8         $this->color = $clr;
9     }
10
11     public function intro(){
12         echo "Printing a message in the parent class <br>";
13         echo "Object Fruit - Name: ".$this->name." Color: ".$this->color;
14     }
15 }
16
17 class Mango extends Fruit {
18     public function msg(){
19         echo "Printing a message in the child class <br>";
20     }
21 }
22
23 $mango = new Mango("Mango","Yellow");
24 $mango->msg();
25 $mango->intro();
26 ?>
```

localhost/oop_lec/fruits.php x +

← → ↻ ⓘ localhost/oop_lec/fruits.php

Printing a message in the child class
Printing a message in the parent class
Object Fruit - Name: Mango Color: Yellow

33

6

Static Variables and Methods

OOP in PHP

34

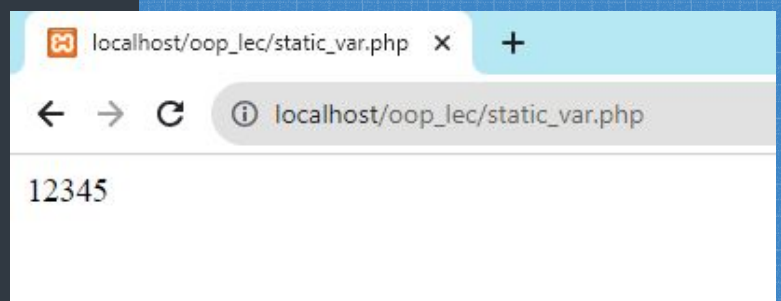
Static Variables

- Constant and Static variables are different but their values are known and fixed in compile time.
- It cannot be changed at runtime, unlike constant, static variable values can be change during runtime.

35

Sample code – static_var.php

```
static_var.php
1  <?php
2      function printStaticVal(){
3          static $cnt = 1;
4          echo $cnt++;
5      }
6
7      printStaticVal();
8      printStaticVal();
9      printStaticVal();
10     printStaticVal();
11     printStaticVal();
12  ?>
```



36

Static Variables

- Remember, if a static and a constant variable is used inside a class it belongs to the class rather than the instance of the class.
- Meaning we can directly access the static and the constant variable without using the created instance of the class.

37

Static Variables

- To access static variable (outside or inside the class), you need to use the **class name** or the **self** keyword (if inside the class).
- Followed by the scope resolution operator `::` and the name of the static variable.
- Ex.

38

Static Variables

- Ex.

```
class Computer{  
    static $ip_add = "192.168.1.100";  
    const os="Windows 11";  
}  
echo Computer::$ip_add;  
echo Computer::os;
```

39

Static Methods

- Static methods have the same concept of Static variables but in a function way.
- We can also use the **parent** keyword when accessing static variables/methods inside a child class.

40

End

Any questions?

41

Activity 1

Tracing

42

1

```
1  <?php
2  class Vehicle {
3      private $brand;
4      private $model;
5      private $year;
6
7      public function __construct($brand, $model, $year) {
8          $this->brand = $brand;
9          $this->model = $model;
10         $this->year = $year;
11     }
12
13     public function __destruct() {
14         echo "Brand: " . $this->brand . "<br>";
15         echo "Model: " . $this->model . "<br>";
16         echo "Year: " . $this->year . "<br>";
17     }
18 }
19 $car = new Vehicle("Ford", "F-150", 2020);
20 ?>
```

2

```
1  <?php
2  class Vehicle {
3      private $brand;
4      private $model;
5      private $year;
6
7      public function __construct($brand, $model, $year) {
8          $this->brand = $brand;
9          $this->model = $model;
10         $this->year = $year;
11     }
12
13     public function __destruct() {
14         echo "Brand: " . $this->brand . " ";
15         echo "Model: " . $this->model . " ";
16         echo "Year: " . $this->year . " ";
17     }
18 }
19 $car = new Vehicle("Ford", "F-150", 2020);
20 ?>
```

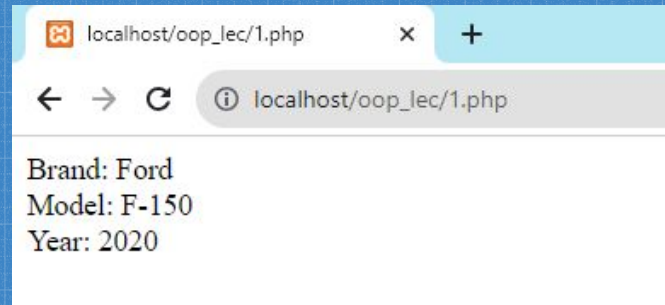
3

```
1 <?php
2 class Vehicle {
3     private $brand;
4     private $model;
5     private $year;
6
7     public function __construct($brand, $model, $year) {
8         $this->brand = $brand;
9         $this->model = $model;
10        $this->year = $year;
11    }
12
13    public function __destruct() {
14        echo "Brand: " . $this->brand . "<br>";
15        echo "Model: " . $this->model . "<br>";
16        echo "Year: " . $this->year . "<br>";
17    }
18 }
19
20 class Ford extends Vehicle{
21     public $price = "P 6,200,000";
22     public function showPrice(){
23         echo "Price: ".$this->price."<br>";
24     }
25 }
26 $car = new Ford("Ford", "F-150", 2023);
27 $car->year = 2024;
28 $car->showPrice();
29 ?>
```

Activity 1

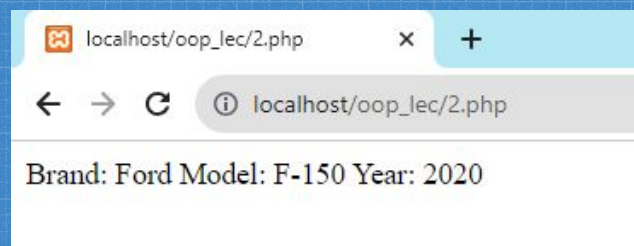
Output

Output # 1



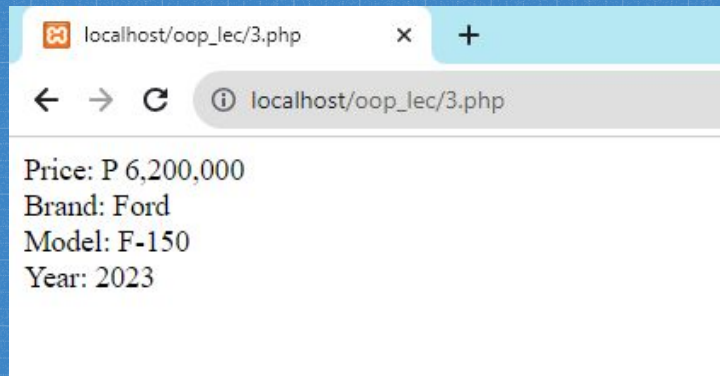
47

Output # 2



48

Output # 3



49

References

- W3Schools (2022). PHP Tutorial. <https://www.w3schools.com/php/default.asp>
- w3resource (2020, February 26). PHP Tutorials for beginners. <https://www.w3resource.com/php/php-home.php>

50